# User Story A4 – Sensors #1: MQ135 Gas Sensor with ADS1115

## Table of Contents

## Task according to work order

In this User Story we have to:

- Solder pin headers to the MQ135 gas sensor module
- Solder pin headers to the ADS1115 16-bit ADC converter module
- Set up the circuit according to the wiring diagram on Digilab breadboard or designated slotson the circuit board
- Determine and configure the I2C address of the ADS1115
- Perform function test using the sample program (MQ135/test.py)
- Document work steps, materials list, and provide video demonstration

## Materials needed

### Hardware Components

- MQ135 Gas Sensor Module (air quality sensor with 4 pins: VCC, GND, AOUT, DOUT)
- ADS1115 16-bit ADC Module (analog-to-digital converter with I2C interface)
- Male pin headers (4-pin for MQ135, 10-pin for ADS1115)
- Raspberry Pi (with GPIO pins available)
- Breadboard or custom PCB with designated sensor slots
- Jumper wires (various colors: red for VCC, black for GND, green for SCL, purple for SDA,blue for analog)

### Tools

- Soldering iron (fine tip, ~350°C)
- Solder wire (rosin core, lead-free recommended)
- Wire cutters / flush cutters
- Helping hands or PCB holder
- Multimeter (for continuity testing)

### Software

- Raspberry Pi OS with Python 3
- I2C tools package (`i2c-tools`)
- Sample test program (MQ135/test.py)

# Processing with the User story

## Checking the necessary components

The first step was to verify that all required modules and components were available and in good condition.

**MQ135 Gas Sensor**: This is an analog air quality sensor that detects various gases including NH3, NOx, benzene, smoke, and CO2. The module has 4 pins:

- **VCC**: 5V power supply (or 3.3V in this configuration)
- **GND**: Ground reference
- **AOUT**: Analog output (variable voltage based on gas concentration)
- **DOUT**: Digital output (threshold-based detection)

**ADS1115 Module**: This is a 16-bit analog-to-digital converter with I2C interface. It is essential because the Raspberry Pi does not have built-in ADC capability and cannot read analog voltages directly. The module provides 4 analog input channels (A0-A3) and communicates via I2C protocol on GPIO pins 2 (SDA) and 3 (SCL).

I inspected both modules for any physical damage and confirmed the pin labels were clearly visible for proper soldering orientation. The components appeared to be in excellent condition.

## Soldering pin headers to the MQ135 module

For this step, I needed to solder a 4-pin male header to the MQ135 sensor module to allow easy connection to the breadboard or PCB.

**Process**: 1. I inserted the 4-pin header through the holes on the MQ135 PCB, ensuring the longer pins protruded from the bottom 2. I used helping hands to hold the module steady at a comfortable 45-degree angle 3. I set the soldering iron to approximately 350°C 4. I applied solder to each pin junction, creating a clean conical solder joint 5. I inspected each joint to ensure proper wetting and mechanical strength 6. I used the multimeter in continuity mode to verify all connections

**What I paid attention to**: The MQ135 sensor contains sensitive gas-detecting elements that can be damaged by excessive heat. I made sure to solder quickly (under 3 seconds per pin) and allowed cooling time between joints. I also avoided applying pressure to the sensor dome on top of the module, and I kept the soldering iron tip clean throughout the process.

## Soldering pin headers to the ADS1115 module

The ADS1115 module required soldering a 10-pin header strip for all available connections (VDD, GND, SCL, SDA, ADDR, ALRT, A0, A1, A2, A3).

**Process**: 1. I positioned the 10-pin male header through the ADS1115 PCB holes 2. I tacked two opposite corner pins first to hold the header straight and perpendicular to the board 3. I checked the alignment with a multimeter to ensure perpendicularity 4. I soldered the remaining pins systematically from left to right 5. I used the flush cutters to trim any excess pin length on the top side if needed 6. I verified all connections with continuity testing

**What I paid attention to**: The ADS1115 is an IC-based module, so heat sensitivity was a concern. I ensured each solder joint was completed quickly with adequate solder flow. I verified there were no solder bridges between adjacent pins, especially on the closely-spaced SCL/SDA and address pins (A0, A1, A2, A3), as these would cause I2C communication failures or incorrect module addressing.

## Assembling the circuit on the breadboard/PCB

According to the wiring diagram provided in the assignment, I needed to connect the modules according to the following connections:

**ADS1115 to Raspberry Pi (I2C Connection)**:

- VDD (ADS1115) → 3.3V (RPi Pin 1) - *Red wire*
- GND (ADS1115) → GND (RPi Pin 9) - *Black wire*
- SCL (ADS1115) → GPIO 3 (RPi Pin 5) - *Green wire*
- SDA (ADS1115) → GPIO 2 (RPi Pin 3) - *Purple wire*

**MQ135 to ADS1115 (Analog Connection)**:

- AOUT (MQ135) → A0 (ADS1115) - *Blue wire*

**MQ135 Power Supply**:

- VCC (MQ135) → 3.3V (RPi Pin 17) - *Red wire*
- GND (MQ135) → GND (RPi Pin 20) - *Black wire*

I carefully routed the jumper wires to avoid crossing and potential interference, using the color-coding scheme to match the reference diagram. I double-checked each connection with the multimeter in continuity mode before powering on the system.

**Important note**: The MQ135 typically operates at 5V and consumes around 150mA, but in this configuration we are using 3.3V from the Raspberry Pi. This may slightly affect sensitivity but prevents potential damage to the 3.3V-tolerant I2C pins.

## Determining the I2C address of the ADS1115

Before running the test program, I needed to verify that the Raspberry Pi could detect the ADS1115 on the I2C bus and determine its address.

**Process**: 1. I first enabled the I2C interface on the Raspberry Pi using `sudo raspi-config` 2. I navigated to: **Interfacing Options → I2C → Enable** 3. After rebooting, I installed the I2C tools: `sudo apt-get install i2c-tools` 4. I ran the I2C detection command: `sudo i2cdetect -y 1`

**Result**: The output showed the ADS1115 at address **0x48** (default address when ADDR pin is unconnected or grounded). This is the standard default I2C address for the ADS1115 module.

I documented this address (0x48) as it needed to be configured in the test program's configuration file.

### Running the test program

With the hardware connected and I2C communication verified, I proceeded to test the sensor using the provided sample program.

**Process**: 1. I navigated to the project directory containing MQ135/test.py 2. I verified the I2C address in the configuration file matched 0x48 3. I ran the test program: `python3 MQ135/test.py`

**Expected behavior**: The program should initialize the ADS1115, read the analog voltage from channel A0 (connected to MQ135 AOUT), and display air quality readings in PPM (parts per million) or raw voltage values.

**Observation**: The program started successfully and began reading sensor values. The output updated at regular intervals (typically every 1-2 seconds).

**Important note**: The MQ135 sensor requires a warm-up period of 24-48 hours for accurate readings after first use, and a pre-heating time of 3-5 minutes before each measurement session.

### Testing results and sensor readings

After running the test program, I observed the sensor output values updating in real-time.

**Observations**:

- Initial readings showed fluctuating values as the sensor warmed up (expected behavior in first 5 minutes)
- After the warm-up period, readings began to stabilize around a baseline value
- Breathing near the sensor caused noticeable increases in the readings
- Introducing a gas source (alcohol hand sanitizer vapor) caused significant changes in the output values
- The analog voltage varied between approximately 0.5V to 2.5V depending on air quality
- The sensor demonstrated proper functionality and responsive behavior to environmental changes

I recorded the sensor readings over a 10-minute period and documented the response times and value ranges.

**Video demonstration**: A video was recorded showing the functioning circuit with real-time sensor value changes and the console output displaying updated measurements.

## What I could improve

**Soldering technique**: While my solder joints were functional, some joints on the ADS1115 could have been more uniform in appearance. With more practice, I could achieve more consistent conical shapes on all pins and avoid slight imperfections on some joints.

**Wire management**: The jumper wire routing on the breadboard was somewhat cluttered. I could improve this by using shorter wires cut to precise lengths and routing them more systematically along the breadboard edges to create a cleaner, more professional appearance.

**Sensor calibration**: I did not perform a full calibration of the MQ135 sensor against known gas concentrations. For more accurate measurements in future projects, I should research proper calibration procedures and test with reference gas samples if available.

**Testing environment**: I conducted testing in a normal indoor environment. For better demonstration of sensor functionality, I could have prepared controlled test conditions with known gas sources to show more dramatic and consistent value changes.

**Documentation during process**: I could have taken more intermediate photos during the soldering process to better document techniques and potential pitfalls for future reference. This would provide a complete visual record of the assembly process.

**Testing duration**: A longer-term test over several hours or days would help verify the stability and drift characteristics of the sensor under various environmental conditions.